

URBANGO — Assignment 1

Course: DSE ZG505 – Preprocessing for AI-Ready Data

Total Marks: 20

Business Scenario

You are a Data Engineer at UrbanGo, an urban ride-hailing platform. UrbanGo wants to prepare operational customer data for a future Customer Churn prediction system.

- **Target Variable:** 1 = Customer Churn, 0 = Customer Retained.
- **Problem Type:** Binary Classification.
- **Dataset Structure:** The dataset contains 13 columns (numerical variables such as distance_km, fare_amount, customer_age, lifetime_rides, customer_rating; and categorical variables such as payment_type, city).
- **Data Quality:** The data has been collected from raw operational systems and is highly noisy. It contains incomplete records, exact duplicate rows, case-sensitive string inconsistencies, physical impossibilities, widening right-tail distributions, and severe global/contextual outliers.

Dataset File Usage & Prerequisites

Initialization: Run the Assignment_A1 program supplied by the instructor. This program generates a student-specific CSV file named <student_email>.csv and a manifest file ~/workspace/A1_execution_manifest.json inside your workspace folder.

Rules: Use your assigned RAW dataset as the input. Do not rename, modify, replace, or manually recreate the assigned RAW dataset or the manifest file.

Prerequisites: Ensure Python 3 is installed along with pandas, scikit-learn, and ydata-profiling.

Dataset Selection & Pipeline Flow (The In-Memory Workflow)

To ensure an end-to-end data engineering pipeline, you must follow standard in-memory DataFrame practices:

1. **The Raw Input:** Load your student-specific raw dataset (<student_email>.csv) into a Pandas DataFrame.
2. **In-Memory Pipeline Operations:** Throughout Part 2, apply all sequential transformations directly to your active DataFrame variable *in-memory*. Do not export, save, or overwrite any CSV files on disk during intermediate steps.
3. **The Clean Output:** At the very end of your pipeline (Step 3.1), export your finalized in-memory DataFrame into a new CSV file named exactly UrbanGo_A1_cleaned.csv. **All data must be saved in the workspace folder because it is persistent memory.**
4. **The ML Evaluation:** In Part 3, the provided execution binary (Assignment_1_part_3) will read all the files stored in the workspace folder to securely ingest both your untouched raw dataset and your newly created clean dataset to evaluate your pipeline's downstream impact.

Part 1: (Total Marks: 7)

Requirement: You must submit your Python code alongside the output/screenshots for every question. Load your student-specific Raw Dataset (<student_email>.csv) into a pandas DataFrame to begin.

1.1 (2 Marks)

Write Python code to calculate and print the following four specific data-quality metrics on your raw dataset. *You must identify the appropriate programmatic methods to extract these counts:*

Note: Marking Scheme: 0.5 marks awarded per correctly coded and printed metric.

1. Calculate the overall percentage of non-null values across the entire dataset.
 2. Count the exact number of rows where customer_age contains biologically impossible values (e.g., <18 or >100).
 3. Count the number of distinct casing variations present in the payment_type column.
 4. Count the total number of exact duplicate rows across the dataset.
- *Marking Scheme: 0.5 marks awarded per correctly coded and printed metric.*

1.2 (1 Mark)

To ensure Data Governance, AI-Ready datasets must document their lineage and structural composition.

- **Task:** Programmatically generate and print a Python dictionary (or JSON object) that acts as a schema metadata log. It must map every column name to its exact data type and its total count of missing values.

1.3 (2 Marks)

Generate an automated profiling report to inspect the data distribution.

- **Task A:** Use ydata-profiling to generate an HTML report of your dataset. Save and submit this HTML file (profiling_report.html).
- **Task B:** Write four distinct technical observations as comments in your code. At least two observations **must** explicitly analyze the asymmetry (skewness) and tail heaviness/extreme outlier presence (kurtosis) of specific numerical columns.

1.4 (2 Marks)

A junior analyst proposes filling every missing numerical value with the arithmetic average. Prove mathematically why this is dangerous for demographic attributes.

Note: Marking Scheme: 1.0 mark for correctly coding and printing the before/after variance; 1.0 mark for the comment explaining variance compression and the disruption of distinct clusters.

- **Task:** Write code to calculate and print the variance of customer_age with its missing values ignored. Then, temporarily fill all missing customer_age values with the arithmetic average, and calculate/print the variance again. Write a one-sentence comment explaining the resulting mathematical effect.

Part 2: (Total Marks: 10)

Requirement: Transform your active DataFrame variable purely in-memory. Submit code and output prints for all steps. **Do not use to_csv() until you reach Part 3.**

2.1 Handling Missing Data Topologies (4 Marks)

Write code to handle missing values by applying the appropriate statistical or algorithmic strategy based on the descriptions below:

1. **(1 Mark): The customer_rating** feature contains missing values resulting from non-random omissions, where the absence of feedback correlates with customer sentiment. Before imputing missing cells, design a programmatic mechanism to capture and preserve this absence pattern using the best approach for this scenario.
2. **(1 Mark):** The fare_amount column contains missing values and exhibits a heavy right tail (skewness > 1.0) caused by extreme surge pricing. Programmatically fill missing values in this column using the single-variable central tendency metric that remains stable and mathematically unaffected by extreme values.
3. **(2 Marks):** Missing records in customer_age belong to a multi-feature demographic manifold where customer behaviors correlate across multiple numerical attributes. Programmatically impute the missing age values using a local neighborhood approach based on geometric proximity to 3 peer records.
 - *Constraint:* You must mathematically scale the relevant features before applying this technique to prevent magnitude dominance during Euclidean distance calculations, and inverse-transform them afterwards.

2.2 Anomaly Remediation & Distribution Alignment (6 Marks)

Write code to detect, filter, and normalize the anomalies present in the dataset based on the following descriptions:

1. **(1.5 Marks):** Identify and permanently drop rows containing physically or logically impossible operational values—specifically local trip distances exceeding 200 km or billing records recording negative fare amounts.
2. **(1.5 Marks):** Extreme surge fares represent genuine business activity but will distort linear models if left unconstrained. Using the spread between the first and third quartiles, calculate the upper statistical boundary ($Q3 + 1.5 \times IQR$) and programmatically cap all fare values exceeding this threshold down to the upper boundary.
3. **(1.5 Marks):** The lifetime_rides column exhibits an exponentially widening positive tail. Apply a non-linear mathematical transformation to this column to compress the extreme right-tail dispersion and align the feature closer to a bell-shaped distribution.
4. **(1.5 Marks):** An operational metric that is normal in a metropolitan market may represent an extreme anomaly in a smaller tier city. Write code to group the dataset by city, compute the mean distance_km per market, and print the resulting contextual baseline summary.

Part 3: Demonstrating the Downstream Effect (Total Marks: 3)

This section evaluates whether your preprocessing decisions in Part 2 successfully helped a linear optimization algorithm find a better decision boundary.

3.1 Prepare the PREPROCESSED Dataset

Export your finalized, in-memory DataFrame from Part 2 into a new CSV file.

- **Mandatory Naming Convention & Location:** Save the file exactly as UrbanGo_A1_cleaned.csv inside your workspace folder. This is strictly required because it is non-volatile memory.

3.2 Execute Assignment_1_part_3

Run Assignment_1_part_3 (supplied by the instructor) from the same workspace. This program performs provenance checks and executes a Logistic Regression comparison.

- **Requirement:** Do not modify, bypass, or decompile the supplied program. The program will automatically read all files stored in the workspace folder (i.e., your original <student_email>.csv and your output UrbanGo_A1_cleaned.csv).

Reference Code (What the Binary Executes Behind the Scenes):

Python

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
```

```
def evaluate_model(X, y):
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42
    )
    model = LogisticRegression(max_iter=100)
    model.fit(X_train, y_train)
    predictions = model.predict(X_test)
    return accuracy_score(y_test, predictions) * 100
```

```
raw_accuracy = evaluate_model(X_raw, y_raw)
```

```
clean_accuracy = evaluate_model(X_clean, y_clean)
```

```
print("=" * 55)
```

```
print("MODEL PERFORMANCE SCORECARD")
```

```
print("=" * 55)
```

```
print(f"Accuracy on RAW Data: {raw_accuracy:.2f}%")
```

```
print(f"Accuracy on PREPROCESSED Data: {clean_accuracy:.2f}%")
```

```
print("=" * 55)
```

3.3 Submission of Output (2 Marks)

Submit the final execution scorecard generated by the program.

- *Marking Scheme:* 2.0 marks awarded for submitting an untampered output showing VERIFIED status and the final accuracy scorecard.

Sample Output:

Plaintext

=====

URBANGO ASSIGNMENT 1 — PART 3 EXECUTION

=====

Student Email : <student email>

Assignment ID : A1

Dataset ID : UG-B

Dataset Fingerprint : 8f4a...c921

Submission ID : A1-XXXXXXX

Verification Status : VERIFIED

MODEL PERFORMANCE SCORECARD

Accuracy on RAW Data : 72.50%

Accuracy on PREPROCESSED Data : 81.25%

Execution Status : SUCCESS

=====

3.4 Technical Synthesis & Conclusion (1 Mark)

- **Task:** Write a 1–2 sentence technical conclusion explaining *why* your preprocessing pipeline improved downstream classification accuracy. and what preprocessing could be added to make performance better?
- Bonus Q (Optional): In terms of its learning, how would you apply these techniques in your day to day worklife?
- *Marking Scheme:* 1.0 mark for correctly explaining that removing impossible sentinels, bounding heavy-tail surge pricing, scaling feature manifolds, and avoiding mean-induced

variance collapse prevented gradient oscillation and allowed the model to converge onto a more accurate decision boundary.

Final Submission Requirements

You must submit:

1. Your Python code (.py or .ipynb) and screenshots of all code outputs for Parts 1 and 2.
2. The generated HTML diagnostic profiling report (profiling_report.html).
3. The final preprocessed dataset (UrbanGo_A1_cleaned.csv) saved in the workspace folder.
4. The final printed scorecard from Part 3 showing VERIFIED status.
5. Your technical observations and written synthesis (from sections 1.3, 1.4, and 3.4).